

06 — BRIN Indexes (Block Range INdexes)

"BRIN: a tiny index for massive tables — when your data is physically ordered."

Table of Contents

1. [Learning Objectives](#)
 2. [BRIN Concept and Design](#)
 3. [ASCII Diagram: BRIN Structure](#)
 4. [Natural Physical Correlation](#)
 5. [When to Use BRIN](#)
 6. [BRIN Operators and Data Types](#)
 7. [Creating BRIN Indexes](#)
 8. [EXPLAIN Output for BRIN Scans](#)
 9. [pages_per_range Configuration](#)
 10. [BRIN vs B-tree Size Comparison](#)
 11. [Autosummarization](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Exercises with Solutions](#)
 17. [Production Scenarios](#)
 18. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain the block range concept and how BRIN summarizes data
 - Identify columns with high natural physical correlation
 - Choose an appropriate `pages_per_range` value for a given table
 - Understand why BRIN is "lossy" and what the Recheck step does
 - Compare BRIN vs B-tree size and query performance tradeoffs
 - Configure autosummarization and manual BRIN maintenance
-

BRIN Concept and Design

BRIN (Block Range INdex) is a radically different approach from B-tree, GIN, and GiST. Instead of indexing individual row values, BRIN summarizes ranges of **heap pages** (block ranges).

Core Idea

Divide the heap file into groups of N consecutive pages (block ranges). For each block range, store the **minimum and maximum** value of the indexed column found in those pages.

```
Block range 1 (pages 0-127):  min=1,  max=128
Block range 2 (pages 128-255): min=129, max=256
```

Block range 3 (pages 256-383): min=257, max=384
...

For a query `WHERE ts BETWEEN '2024-01-01' AND '2024-01-31'`, BRIN checks each block range summary. If a range's `[min, max]` interval doesn't overlap the query range, the entire block range (128 pages × 8 KB = 1 MB) is skipped.

Key Properties

- **Very small:** one entry per block range, not one per row
- **Lossy:** can produce false positives (block ranges may be returned even if no exact match exists inside them)
- **Requires physical correlation:** only effective when column values are physically sorted in heap order (i.e., when large min-max ranges don't overlap between block ranges)
- **Near-zero write overhead:** BRIN doesn't need updating on most INSERTs/UPDATEs
- **Ideal for time-series and monotonically increasing columns**

ASCII Diagram: BRIN Structure

Heap file: events table (10 million rows, inserted in timestamp order)
Each page = 8 KB, ~100 rows per page → ~100,000 pages total

BRIN index (pages_per_range = 128):

Block Range	Pages	min_ts	max_ts
Range 0	0-127	2020-01-01 00:00:00	2020-01-15 23:59:59
Range 1	128-255	2020-01-16 00:00:00	2020-02-01 23:59:59
Range 2	256-383	2020-02-02 00:00:00	2020-02-18 23:59:59
...			
Range 780	99840-99967	2024-06-01 00:00:00	2024-06-15 23:59:59
Range 781	99968-99999	2024-06-16 00:00:00	2024-06-30 23:59:59

Total index entries: ~782 entries (vs ~10 million for B-tree leaf entries)
BRIN index size: ~100 KB (vs ~400 MB for B-tree on timestamp column)

BRIN INDEX (tiny!)

Range 0: [2020-01-01 .. 2020-01-15]
Range 1: [2020-01-16 .. 2020-02-01]
Range 2: [2020-02-02 .. 2020-02-18]
...
Range 781: [2024-06-16 .. 2024-06-30]

Query: `WHERE ts BETWEEN '2024-06-01' AND '2024-06-30'`

Check each range:

Range 0: `[2020-01-01..2020-01-15] ∩ [2024-06-01..2024-06-30] = ∅ → SKIP`

Range 1: `[2020-01-16..2020-02-01] ∩ [2024-06-01..2024-06-30] = ∅ → SKIP`

...

Range 780: [2024-06-01..2024-06-15] ∩ [2024-06-01..2024-06-30] ≠ ∅ → SCAN
Range 781: [2024-06-16..2024-06-30] ∩ [2024-06-01..2024-06-30] ≠ ∅ → SCAN

Only scan pages 99840-99999 (160 pages out of 100,000) = 0.16% of table!

Natural Physical Correlation

BRIN's effectiveness depends entirely on the **correlation** between the indexed column's logical sort order and the physical order of rows in the heap.

pg_stats.correlation

```
-- Check correlation (1.0 = perfect, 0.0 = random, -1.0 = reversed order)
SELECT attname, correlation
FROM pg_stats
WHERE tablename = 'events'
ORDER BY abs(correlation) DESC;
```

Correlation Values and BRIN Effectiveness

Correlation	BRIN Effectiveness	Example Column
~1.0 or ~-1.0	Excellent	created_at (append-only table)
0.8-1.0	Good	id BIGSERIAL (mostly sequential)
0.5-0.8	Moderate	updated_at (updates spread records)
0.2-0.5	Poor	Random values with some clustering
~0.0	Useless	UUID, random values, fully shuffled data

High-Correlation Columns (BRIN is great)

- **Auto-increment IDs:** BIGSERIAL, SERIAL — rows are inserted in order
- **Insert timestamps:** created_at — time increases as rows are inserted
- **Log sequence numbers:** monotonically increasing
- **IOT (Index-Organized Table) data:** data written in bulk by date/region

Low-Correlation Columns (BRIN is poor)

- **UUIDs:** random, no correlation
- **Status columns:** updated frequently, rows scattered
- **Columns updated in random order**

When to Use BRIN

Ideal Scenarios

1. **Very large time-series tables** (logs, events, metrics, IoT data)
 - Table: billions of rows

- Column: `created_at` or `timestamp` (monotonically increasing)
- Access pattern: queries for recent data or specific time windows

2. Append-only tables with sequential IDs

- Column: `id BIGSERIAL`
- Access: range queries on id (e.g., batch processing)

3. Data warehouse fact tables partitioned by date

- Even without partitioning, BRIN provides cheap partition-like skipping

4. When storage is extremely constrained

- BRIN on a billion-row table: ~10 MB
- B-tree on same table: ~50 GB

Decision Rule

```
Table size > 100M rows?
AND column has correlation > 0.8?
AND queries are range-based (WHERE ts BETWEEN ...)?
AND you can tolerate occasional false-positive block scans?
→ Use BRIN

Otherwise → Use B-tree
```

BRIN Operators and Data Types

Supported Types

BRIN has operator classes for most built-in scalar types:

Type	Operator Class	Operators
<code>int2/4/8, float4/8</code>	<code>int4_minmax_ops</code> , etc.	<code>=, <, >, <=, >=, BETWEEN</code>
<code>text, varchar</code>	<code>text_minmax_ops</code>	<code>=, <, >, <=, >=</code>
<code>date, timestamp, timestamptz</code>	<code>timestamp_minmax_ops</code>	<code>=, <, >, <=, >=, BETWEEN</code>
<code>uuid</code>	<code>uuid_minmax_ops</code>	Same (but correlation usually near 0)
<code>inet, cidr</code>	<code>inet_minmax_ops</code>	Same

Multi-Range / Bloom BRIN (PostgreSQL 14+)

PostgreSQL 14 introduced additional BRIN operator classes:

- `int4_minmax_multi_ops` : stores multiple min/max ranges per block range (better for data with some correlation but not perfect)
- `float8_bloom_ops` : bloom filter per block range for equality checks

```
-- Multi-range BRIN (PG14+): better for partially-correlated data
CREATE INDEX idx_orders_amount_brin ON orders
USING BRIN (amount int4_minmax_multi_ops);
```

Creating BRIN Indexes

```
-- Basic BRIN index (default pages_per_range = 128)
CREATE INDEX idx_events_ts ON events USING BRIN (created_at);

-- Custom pages_per_range
CREATE INDEX idx_events_ts ON events USING BRIN (created_at)
WITH (pages_per_range = 64);

-- Concurrent (minimal locking)
CREATE INDEX CONCURRENTLY idx_events_ts ON events USING BRIN (created_at);

-- Multi-column BRIN
CREATE INDEX idx_events_multi ON events
USING BRIN (tenant_id, created_at);

-- With autosummarize enabled (auto-updates summaries for new pages)
CREATE INDEX idx_events_ts ON events
USING BRIN (created_at) WITH (autosummarize = on);

-- Check existing indexes
SELECT indexname, pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE relname = 'events';
```

EXPLAIN Output for BRIN Scans

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM events
WHERE created_at BETWEEN '2024-06-01' AND '2024-06-30';
```

```
Aggregate  (cost=18234.56..18234.57 rows=1 width=8)
    (actual time=123.456..123.456 rows=1 loops=1)
    Buffers: shared hit=1567
    -> Bitmap Heap Scan on events  (cost=52.23..18123.45 rows=44444 width=0)
        (actual time=1.234..119.234 rows=43210 loops=1)
        Recheck Cond: ((created_at >= '2024-06-01') AND (created_at <= '2024-06-30'))
        Rows Removed by Recheck: 234          ← false positives from BRIN
        Heap Blocks: lossy=256                ← "lossy" = BRIN block ranges, not
exact
        Buffers: shared hit=1567
        -> Bitmap Index Scan on idx_events_ts
```

```
(cost=0.00..41.12 rows=44444 width=0)
(actual time=0.567..0.567 rows=0 loops=1)  ← 0 = uses blocks not rows
Index Cond: ((created_at >= '2024-06-01') AND (created_at <= '2024-06-
30'))
Planning Time: 0.234 ms
Execution Time: 123.890 ms
```

Key indicators of BRIN scan:

- `Heap Blocks: lossy=N` — the word "lossy" confirms BRIN (not B-tree)
- `Rows Removed by Recheck: N` — false positives from BRIN's lossy summaries
- `Bitmap Index Scan rows=0` — BRIN returns block ranges, not row counts

Compare to full sequential scan (no index):

```
Seq Scan on events (cost=0..89234.00 rows=44444 width=...)
(actual time=0.023..4567.234 rows=43210 loops=1)
Buffers: shared hit=100000 ← ALL pages read!
```

pages_per_range Configuration

`pages_per_range` is the most important tuning parameter for BRIN.

Trade-offs

pages_per_range	Block range size	Index size	Selectivity	False positives
16	128 KB	Larger	High (precise)	Few
64	512 KB	Moderate	Moderate	Moderate
128 (default)	1 MB	Small	Good	Some
256	2 MB	Very small	Lower	More
512	4 MB	Tiny	Low	Many

Choosing pages_per_range

```
-- Step 1: Check table size
SELECT pg_size_pretty(pg_relation_size('events'));

-- Step 2: Check typical query range size
-- If queries typically cover 1 week out of 2 years:
-- 1 week / 2 years ≈ 1% selectivity
-- You want BRIN to identify ~1% of blocks

-- Step 3: Calculate optimal pages_per_range
-- total_pages / (expected_matching_pages_per_query × some_factor)
-- e.g., 100,000 pages, 1% queries → should skip 99%
-- pages_per_range = 128 means 782 ranges for 100K pages
-- A 1-week query would match ~7 ranges (7/782 = ~0.9% of index entries)
```

```
-- This is reasonable.

-- Step 4: Test with EXPLAIN ANALYZE
-- Tune until "Heap Blocks: lossy=" shows reasonable count
```

Autosummarization

When rows are inserted into a table with a BRIN index, new pages are initially **unsummarized**. BRIN must summarize them before it can skip them.

Behavior Without Autosummarize

```
-- autosummarize = off (default in older PostgreSQL)
-- New pages: not summarized → treated as "might match" → always scanned
-- The index grows stale as new data is inserted

-- Manual summarization:
SELECT brin_summarize_new_values('idx_events_ts');
-- or
SELECT brin_summarize_range('idx_events_ts', <first_block>, <last_block>);
```

Behavior With Autosummarize (PostgreSQL 10+)

```
-- autosummarize = on: autovacuum automatically summarizes new page ranges
CREATE INDEX idx_events_ts ON events
USING BRIN (created_at) WITH (autosummarize = on);

-- Check summarization status
SELECT * FROM pg_brin_page_items(get_raw_page('idx_events_ts', 2), 'idx_events_ts');
```

Recommendation

Always set `autosummarize = on` for append-heavy tables to keep BRIN effective for recently-inserted data.

BRIN vs B-tree Size Comparison

For a table with 100 million rows and a `created_at` TIMESTAMPTZ column:

B-tree index:

Leaf entries: $100M \times (8 \text{ bytes timestamp} + 6 \text{ bytes ctid} + 2 \text{ bytes overhead}) = \sim 1.5 \text{ GB}$
Internal pages: $\sim 15 \text{ MB}$
Total: $\sim 1.5 \text{ GB}$

BRIN index (pages_per_range = 128):

Heap pages: $\sim 100M \text{ rows} \times 100 \text{ bytes avg} / 8192 \text{ bytes} = \sim 1.2M \text{ pages}$
Block ranges: $1.2M / 128 = \sim 9,375 \text{ ranges}$
Per range: $8+8 \text{ bytes (min+max timestamps)} + \text{overhead} = \sim 32 \text{ bytes}$
Total: $9,375 \times 32 \text{ bytes} = \sim 300 \text{ KB}$

Size ratio: B-tree is ~5,000× larger than BRIN!

Query Performance Comparison

For query `WHERE created_at = '2024-06-15 10:00:00'` (exact timestamp, ~100 matching rows):

Index	Pages Read	Query Time
No index	~1,200,000	~45s
BRIN	~128 (one block range, with false positives)	~0.5s
B-tree	~5	~0.02s

For exact equality or very narrow ranges, B-tree is still much faster than BRIN. BRIN shines for wide range queries that select many pages.

Common Mistakes

1. Using BRIN on unordered columns

```
-- WRONG: UUID has near-zero correlation
CREATE INDEX ON orders USING BRIN (id) -- id is UUID
-- BRIN will be as slow as a sequential scan; every block range matches
```

2. Not checking correlation before creating BRIN

```
-- Always verify first:
SELECT correlation FROM pg_stats
WHERE tablename = 'events' AND attname = 'created_at';
-- correlation < 0.8 → BRIN not recommended
```

3. Using BRIN for selective queries expecting B-tree speed

```
-- BRIN is NOT a replacement for B-tree on selective queries
-- WHERE created_at = '2024-06-15 10:00:00.000'
-- B-tree: 5 page reads
-- BRIN: 128+ page reads (entire block range)
-- For narrow queries, B-tree is 25×+ faster
```

4. Forgetting autosummarize on append-heavy tables

Without `autosummarize`, new data pages are never summarized and always scanned.

5. Setting `pages_per_range` too small

A very small `pages_per_range` (e.g., 4) makes BRIN nearly as large as a B-tree but without B-tree's precision or flexibility — worst of both worlds.

Best Practices

1. Use BRIN for very large tables (>100M rows) with high-correlation columns.
2. Check `pg_stats.correlation` before creating BRIN — if below 0.8, use B-tree.
3. Enable `autosummarize = on` for tables with ongoing inserts.
4. Use BRIN alongside B-tree for different query patterns: BRIN for range queries (large time windows), B-tree for selective lookups (specific rows).
5. Test `pages_per_range` — the default of 128 is a good starting point for most tables, but adjust based on your query selectivity.
6. Consider table partitioning as a complement or alternative to BRIN — monthly partitions achieve similar block-skipping with better selective query performance.
7. Never use BRIN as the sole index on a column that needs exact-match performance — BRIN's block-range granularity is too coarse for high-selectivity point queries.

Performance Considerations

BRIN Maintenance Cost

BRIN has essentially zero maintenance cost on inserts: new entries just mark the new page range as "unsummarized" (deferred until autovacuum or manual summarization). This makes it ideal for high-throughput insert workloads.

BRIN Update on Writes

Unlike B-tree which must be updated synchronously on every write, BRIN updates are deferred. This reduces write latency significantly for large tables.

Combined Index Strategy

```
-- For a large events table:
-- B-tree for recent, selective queries (user_id, event_type lookups)
CREATE INDEX CONCURRENTLY idx_events_user ON events(user_id, created_at);

-- BRIN for time-range analysis (large time windows)
CREATE INDEX CONCURRENTLY idx_events_ts_brin ON events
USING BRIN (created_at) WITH (autosummarize = on);

-- B-tree is used for: WHERE user_id = 42 ORDER BY created_at DESC LIMIT 10
-- BRIN is used for: WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31'
```

Interview Questions & Answers

Q1. What is a block range in a BRIN index and what information is stored per range?

A: A block range is a group of consecutive heap pages (e.g., 128 pages = 1 MB by default). For each block range, BRIN stores a summary of the indexed column values within those pages. For scalar types (integers, timestamps, etc.) the summary is the minimum and maximum value found in the range. For multi-range variants (PG14+), multiple min/max pairs can be stored. This summary is used to quickly determine whether a block range can possibly contain rows matching a query predicate — if the query range doesn't overlap the [min, max] summary, the entire block range is skipped.

Q2. What is "physical correlation" and why does it determine BRIN's effectiveness?

A: Physical correlation is the statistical relationship between a column's logical sort order and the physical order of rows in the heap. A correlation of 1.0 means rows with larger column values are always stored in later heap pages — the column increases monotonically as you scan the heap file. BRIN is only effective when correlation is high (>0.8) because: if rows are physically ordered by the column, each block range's [min, max] interval is narrow and non-overlapping with adjacent ranges. A query for a specific time window matches only a few block ranges. If correlation is low (rows are randomly distributed), every block range's min/max would span nearly the entire value domain, and the BRIN would match almost every block range — providing no benefit over a sequential scan.

Q3. Why does EXPLAIN ANALYZE show "Heap Blocks: lossy=N" for BRIN but "Heap Blocks: exact=N" for other indexes?

A: "Lossy" means the bitmap represents entire block ranges (groups of pages), not individual rows. BRIN doesn't know exactly which rows within a block range match — only that the range's [min, max] summary overlaps the query. So the bitmap contains bits for entire block ranges, not individual row locations. The "exact" label appears with B-tree and GIN because those indexes identify specific ctids (row locations), allowing the bitmap to represent exact rows. The lossy bitmap causes the heap scan to read more pages and apply a Recheck condition to filter out false positives.

Q4. Compare BRIN and B-tree index sizes for a timestamp column on a billion-row table.

A: For a billion-row table with a timestamp column:

- B-tree: ~1 billion leaf entries × ~16 bytes each = ~15 GB index
- BRIN (128 pages_per_range, ~10M heap pages): $10M/128 = \sim 78,125$ ranges × ~32 bytes = ~2.5 MB

BRIN is roughly 6,000× smaller. For a single server with 128 GB RAM, a 15 GB B-tree index would consume significant buffer cache, while a 2.5 MB BRIN index fits entirely in cache at negligible cost. The tradeoff: BRIN is much slower for selective queries (a specific timestamp match reads 128 pages vs 3-4 for B-tree).

Q5. When would you use both a BRIN and a B-tree index on the same column?

A: This makes sense when you have two distinct query patterns: (1) Analytical range queries that scan large time windows (e.g., monthly aggregations) — BRIN efficiently skips most of the table. (2) Operational point queries that need specific rows quickly (e.g., `WHERE created_at = ?` or very narrow ranges) — B-tree provides precise 3-4 page lookups. The planner will choose B-tree for narrow predicates and BRIN for wider range queries based on estimated selectivity. Both indexes together consume far less space than two B-trees.

Exercises with Solutions

Exercise 1

A `sensor_readings` table has 500 million rows inserted in timestamp order. Design a BRIN index and demonstrate why it's appropriate.

Solution:

```
-- Check correlation first
ANALYZE sensor_readings;
SELECT attname, correlation
FROM pg_stats WHERE tablename = 'sensor_readings' AND attname = 'recorded_at';
-- Expected: correlation ≈ 1.0 (timestamps are insert-ordered)

-- Create BRIN
CREATE INDEX CONCURRENTLY idx_sensor_ts
ON sensor_readings USING BRIN (recorded_at)
WITH (pages_per_range = 128, autosummarize = on);

-- Check size vs hypothetical B-tree
SELECT pg_size_pretty(pg_relation_size('idx_sensor_ts')) AS brin_size;
-- Expected: ~5 MB vs ~8+ GB for B-tree

-- Verify usage
EXPLAIN (ANALYZE, BUFFERS)
SELECT AVG(value) FROM sensor_readings
WHERE recorded_at BETWEEN '2024-01-01' AND '2024-01-31';
-- Should show Bitmap Index Scan with "lossy" blocks
```

Production Scenarios

Scenario 1: IoT Data Warehouse

An IoT platform ingests 10 million sensor readings per day. The table has grown to 10 billion rows over 3 years. Monthly analytics queries (e.g., average temperature per hour for a given month) were timing out.

```
-- Problem: B-tree on timestamp would be 150+ GB
-- BRIN solution:
CREATE INDEX CONCURRENTLY idx_iot_ts ON sensor_readings
USING BRIN (reading_time) WITH (autosummarize = on);

-- Size: ~25 MB (vs 150+ GB for B-tree)

-- Monthly analytics (uses BRIN efficiently):
SELECT date_trunc('hour', reading_time) AS hour,
       AVG(temperature) AS avg_temp
FROM sensor_readings
WHERE reading_time BETWEEN '2024-06-01' AND '2024-06-30'
      AND sensor_id = 12345
GROUP BY hour ORDER BY hour;

-- Result: Query went from timeout (>5 min) to 3 seconds
-- BRIN skips 11/12 months of data (only June ranges scanned)
```

Cross-References

- [01_index_fundamentals.md](#) — Index types overview
- [02_btree_indexes.md](#) — B-tree for comparison
- [12_index_maintenance.md](#) — BRIN summarization maintenance
- [../08_Query_Optimization/06_statistics_and_estimates.md](#) — pg_stats.correlation